

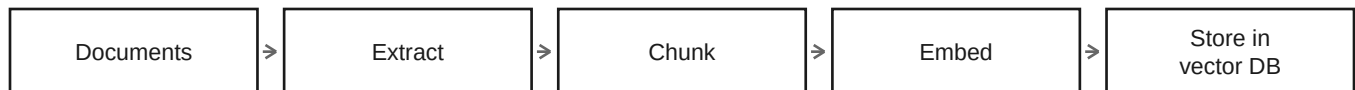
RAG Explained

Day 19 — RAG Fundamentals, practical task

RAG stands for Retrieval-Augmented Generation. The basic idea is that a plain LLM only knows what it saw in training, so it has no idea what's in your own documents. RAG fixes that by looking up relevant pieces of your own data first, then handing that to the LLM as context before it answers. That's what makes the answer "grounded" instead of a guess, and it also means the system can say "I don't know" when nothing relevant turns up, instead of making something up.

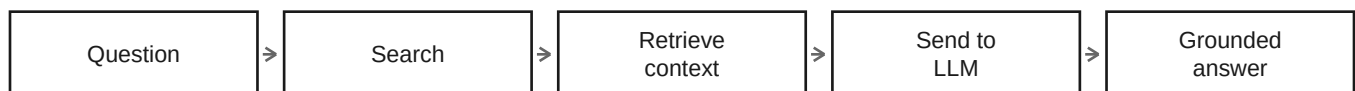
Ingestion path

— happens once, when documents get added



Query path

— happens every time someone asks a question



Chunking, and why the size matters

Chunking is just cutting documents into smaller pieces before embedding them, and it turns out to be the part that makes or breaks a RAG system:

- If chunks are too big, each one covers several topics at once, so a search pulls back a mix of relevant and irrelevant text and the answer gets diluted.
- If chunks are too small, you lose the surrounding context — a single retrieved sentence can be meaningless on its own.
- In practice most people land somewhere around 300-800 tokens per chunk, usually with a bit of overlap between chunks so an idea doesn't get cut in half at the boundary.

3 real problems RAG would solve at a Petalnex-type business

- Proposal writing — new scopes/SOWs could be grounded in real past client proposals instead of being written from a blank page each time.
- Project handoff — a developer picking up someone else's client project could ask questions against that project's own docs and tickets instead of digging through Slack history.
- Sales questions — "does Petalnex do X" could be answered from the real service pages and case studies, with a safe "not sure, let me check" fallback instead of a made-up answer.

Key ideas: embeddings, similarity search, chunking, metadata, grounding.